MAVEN SILICON

Centre of Excellence in Semicon

APB based SPI Protocol

www.maven-silicon.com

Maven Silicon Confidential

All the presentations, books, documents [hard copies and soft copies] labs and projects [source code] that you are using and developing as part of the training course are the proprietary work of Maven Silicon and it is fully protected under copyright and trade secret laws. You may not view, use, disclose, copy, or distribute the materials or any information except pursuant to a valid written license from Maven Silicon

Table of Contents

1. Introduction

The APB-Based Serial Peripheral Interface (SPI) Core is a versatile and efficient module designed for seamless communication between a master device and peripheral devices. The SPI protocol is widely used for its simplicity and high-speed data transfer capabilities. It is an ideal choice for various applications including embedded systems, sensor interfacing, and communication modules.

The SPI Core comprises several key components like status registers to monitor the core's state, control registers to configure and manage its operations, data registers for temporary storage of transmitted or received data, shifter logic to handle serial-to-parallel and parallel-to-serial data conversion, a baud rate generator to control the clock rate for data transmission, and master and slave control logic to manage the operational modes.

The SPI Core supports duplex mode communication for simultaneous two-way data transfer and ensures synchronous communication by transferring data in sync with the clock signal. It complies with the AMBA APB3 protocol, facilitating easy integration into APB-based systems. The core provides control over clock signal characteristics to accommodate different SPI modes (Mode 0, Mode 1, Mode 2, Mode 3) and is capable of operating at high clock frequencies for rapid data transfer. Additionally, it includes interrupt capabilities for efficient event handling and low-latency data transfer. The SPI Core is optimized for low power consumption, making it suitable for battery-operated devices.

Register Address Map:

Address	Register	Access
0	SPI Control Register 1	RW
1	SPI Control Register 2	RW
2	SPI Baud Rate Register	RW
3	SPI Status Register	RO
5	SPI Data Register	RW

SPI Control Register 1:

SPIE	SPE	SPTIE	MSTR	CPOL	CPHA	SSOE	LSBFE

Reset Value = 8'b0000_0100

SPIE - This bit enables SPI to interrupt requests if the SPIF or MODF status flag is set.

SPE - This bit enables the SPI system and dedicates the SPI port pins to SPI system functions.

SPTIE - This bit enables SPI interrupt requests if the SPTEF flag is set.

MSTR – This bit enables the SPI Core to work as a Master.

CPOL - This bit selects an inverted or non-inverted SPI clock.

CPHA -This bit is used to select the SPI clock format.

SSOE - The SS output feature is enabled only in master mode.

LSBFE – Data is transferred the least significant bit.

SPI Control Register 2:

0	0	0	MODFEN	BIDIROE	0	SPISWAI	SPC0

Reset Value = 8'b0000_0000

MODFEN -This bit allows the MODF failure to be detected.

BIDIROE -Output enable in the Bidirectional mode of operation.

SPISWAI - This bit is used for power conservation while in wait mode.

SPC0 – Serial Pin Control Bit 0.

SPI Baud Rate Register:

0	SPPR2	SPPR1	SPPR0	0	SPR2	SPR1	SPR0

Reset Value = 8'b0000_0000

SPPR2 – SPPR0 - SPI Baud Rate Preselection Bits

SPR2 – SPR0 – SPI Baud Rate Selection Bits

SPI Status Register:

SPIF	0	SPTEF	MODF	0	0	0	0

Reset Value: 8'b0010_0000

SPIF - This bit is set after a received data byte has been transferred into the SPI Data Register.

SPTEF - If set, this bit indicates that the transmit data register is empty.

MODF - This bit is set if the SS input becomes low while the SPI is configured as a master and mode fault detection is enabled, MODFEN bit of SPICR2 register is set.

SPI Data Register:

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0

Reset Value = 8'b0000_0000

2. SPI Core

The SPI Core is designed to interface with an APB Master, such as a microcontroller, receiving data from it and communicating serially with peripheral devices. The communication occurs based on the rising or falling edge of the serial clock (SCLK), depending on the Clock Phase (CPHA) and Clock Polarity (CPOL) settings.

Data Reception and Transmission

Data to be transmitted by the SPI Core is received via the APB Bus. Upon writing new data into the data register, the SPI Core automatically initiates data transfer. The data is transmitted through the Master Out Slave In (MOSI) line, while incoming data is received through the Master In Slave Out (MISO) line.

The following sequence outlines the data transfer process:

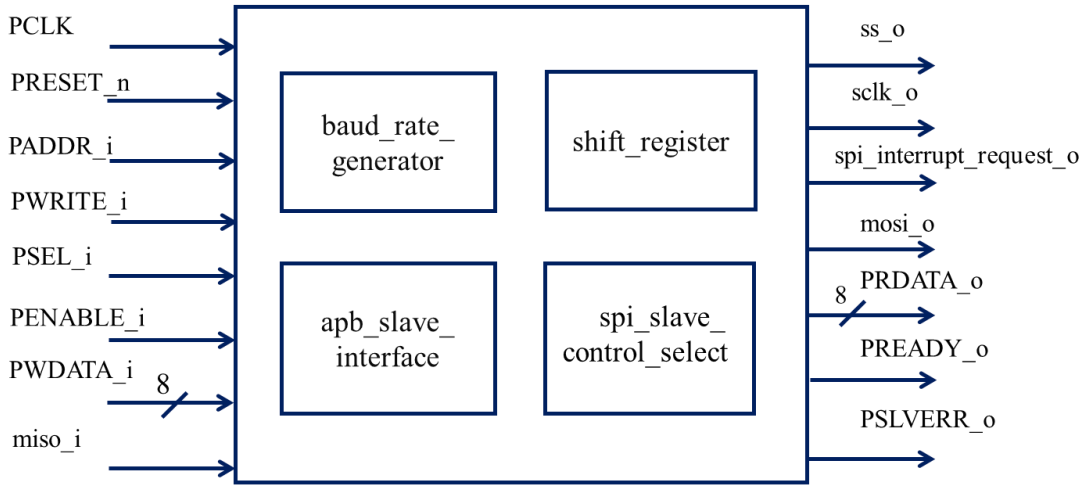
1. **Data Loading**: Data is written into the SPI data register via the APB Bus.
2. **SS Activation**: The SS signal is driven low to initiate communication.
3. **Clock Signal Generation**: The SPI Core generates the SCLK based on the configured CPHA and CPOL settings.
4. **Data Transmission**: Data is shifted out through MOSI and received through MISO on the clock edges.
5. **Transfer Completion**: The SS signal is driven high, and the SPI Core ceases clock generation, marking the end of the data transfer.

By adhering to this protocol, the SPI Core ensures reliable and efficient communication between the master and slave devices in various operational modes and conditions.

The SPI Core consists of the following 4 blocks

- APB Slave Interface
- Baud Rate Generator
- SPI Slave Select Generator
- Shifter

Block Diagram:



Signal Description:

Signal	Width	Direction	Signal Description
PCLK	1	Input	PCLK is a clock signal. All signals are timed against the rising edge of PCLK
PRESET_n	1	Input	Active Low Asynchronus Reset Signal
PADDR_i	3	Input	Address bus APB (3 bits)
PSEL_i	1	Input	APB Slave Select Signal
PENABLE_i	1	Input	PENABLE indictes the second and subsequent APB transfer
PWRITE_i	1	Input	PWRITE indicates an APB write access when High and APB read access when LOW.
PWDATA_i	8	Input	The PWDATA is write data bus.
PRDATA_o	8	Output	The PRDATA is read data bus.
PREADY_o	1	Output	PREADY is used to extend an APB Transfer by Core
PSLVERR_o	1	Output	PSLVERR is used to indicate Transfer Error.
mosi_o	1	Output	MOSI is used to transmit data out of the SPI Core.
miso_i	1	Input	MISO is used to receive data to SPI Core.
ss_o	1	Output	Slave Select is an active low signal. Indicates that slave has been selected when it is low.
sclk_o	1	Output	SCLK is used to output the clock with respect to which the SPI transfers data(Serial Clock)
spi_interrupt_ Request_o	1	Output	SPI Interrupt Request

3. Baud Rate Generator

Objective: *To write synthesizable RTL code for an Arithmetic Baud rate generator of the SPI core and verify with a Verilog-based linear testbench. Validate the RTL coding style using the linting process and generate the gate-level netlist.*

Working Directory : /home/<user_name>/spi_design_lab/

Relevant Directories :

tb : tb source code
rtl : rtl source code
sim : Simulation directory
synth : Synthesis directory
lint : Lint directory

Functionality:

The baud_generator module is designed to generate a serial clock (SCLK) for a Serial Peripheral Interface (SPI) device based on the Advanced Peripheral Bus (APB) clock (PCLK). It calculates the baud rate divisor using SPI Baud Rate Preselection Bits (sppr) and SPI Baud Rate Selection Bits (spr). The clock polarity (cpol) determines the initial state of the clock. The module uses a counter to divide the PCLK by the calculated baud rate divisor, toggling the SCLK each time the counter reaches this value. This ensures the SCLK is generated at the correct frequency for SPI communication.

The module operates in different SPI modes, including run, wait, and stop modes, adjusting the clock generation accordingly. If the SPI is in run mode or wait mode (with spiswai not asserted) and the slave select (ss) is active, the SCLK is toggled based on the baud rate divisor. If the SPI is not selected or in stop mode, the SCLK is reset to its initial polarity state, and the counter is reset. This design allows the baud_generator to produce a stable and accurate SCLK for SPI communication, facilitating efficient data transfer between the SPI master and slave devices.

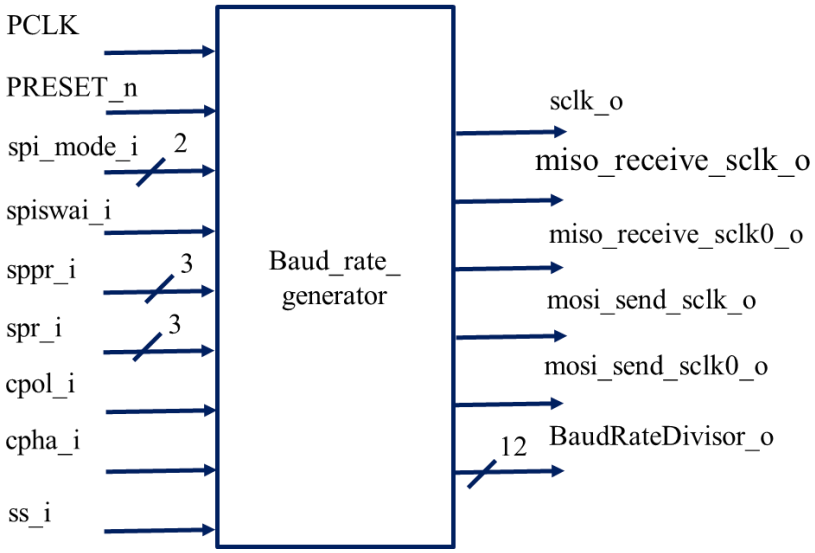
The Baud Rate Divisor is calculated using the following equation:

$$\text{Baud Rate Divisor} = (\text{SPPR}+1) \times 2^{(\text{SPR}+1)}$$

Once the Baud Rate Divisor is determined, the Baud Rate can be calculated using the equation:

$$\text{Baud Rate} = (\text{PCLK}/\text{BAUD RATE DIVISOR})$$

Block Diagram:



Signal Description:

Signal	Width	Direction	Signal Description
PCLK	1	Input	System Clock
PRESET_n	1	Input	Active Low Asynchronus Reset Signal
cpol_i	1	Input	Clock Polarity
spiswai_i	1	Input	SPI Stop in Wait Mode
spi_mode_i	2	Input	SPI Mode(Run Mode, Wait Mode and Stop Mode)
spr_i	3	Input	SPI Baud Rate Selection Bits
sppr_i	3	Input	SPI Baud Rate Preselection Bit
ss_i	1	Input	Slave Select Active Low Signal
sclk_o	1	Output	Serial Clock
BaudRateDivisor_o	12	Output	Baud Rate Divisor
cphase_i	1	Input	Clock Phase
miso_receive_sclk_o _o	1	Output	Flag to indicate when to receive the miso data when both cphase and cpol are high or low.
miso_receive_sclk0_o _o	1	Output	Flag to indicate when to receive the miso data when either of the cphase or cpol is high.
mosi_send_sclk_o _o	1	Output	Flag to indicate when to send the mosi data when both cphase and cpol are high or low.
mosi_send_sclk0 _o	1	Output	Flag to indicate when to send the mosi data when either of the cphase or cpol is high.

Pseudocode:

Define Module and Ports

- Start by naming your module spi_baud_generator.

- Declare **input ports**:
    - System clock and reset.
    - SPI configuration inputs:
        - spi_mode_i → 2-bit input for run, wait, stop modes.
        - spiswai_i → used in wait mode.
        - sppr_i, spr_i → baud rate divisor config.
        - cpol_i, cpha_i → SPI clock polarity and phase.
        - ss_i → slave select.
- Declare **output ports**:
    - sclk_o → generated SPI clock.
    - Flags for sampling **MISO** and transmitting **MOSI**.
    - BaudRateDivisor_o → computed baud rate divisor value.

Declare Internal Signals

- A **wire pre_sclk_s**: Initial polarity for sclk_o.
- A **register count_s**: 12-bit counter for baud rate control.

Compute Baud Rate Divisor

- Implement combinational logic to compute baud rate divisor as: Use operators to implement the below equation.

$$\text{BaudRateDivisor} = (SPPR + 1) \times 2^{(SPR+1)}$$

Generate Initial SCLK Polarity

- **pre_sclk_s** asserted high if cpol_i is true else false

Generate SPI Clock (SCLK)

- Create a sequential block **sensitive to PCLK and active-low reset**.
- If reset is active:
    - Clear counter.
    - Set sclk_o to pre_sclk_s.
- Else if ss_i is low (slave selected) and spiswai_i is low and spi_mode = run or spi_mode wait mode:
        - When counter reaches (BaudRateDivisor_o/2 -1), toggle sclk_o and reset counter

- Else Increment counter.

- Otherwise:
  - Hold sclk_o to initial polarity.
  - Reset counter.

Generate MISO Sample Flags

- Create a **sequential block** for miso_receive_sclk_o and miso_receive_sclk0_o.
- On reset, clear both.
- This condition essentially checks whether the **sampling point for MISO data is on the rising or falling edge of SCLK**, based on CPOL and CPHA.

CPOL	CPHA	Active Sampling Edge
0	0	Rising Edge
0	1	Falling Edge
1	0	Falling Edge
1	1	Rising Edge

- If this (!cpha_i && cpol_i)||(cpha_i && !cpol_i)is true

  If the current SCLK is **high** (logic 1).

  - if count_s has reached the final count (BaudRateDivisor_o/2-1) — meaning one half-period is over.
  - Assert miso_receive_sclk0_o flag for one clock cycle of PCLK else, clear it.
  - Else clear it

- If (!cpha_i && !cpol_i)||(cpha_i && cpol_i) is true

  If the current SCLK is **low** (logic 0).

  - if count_s has reached the final count (BaudRateDivisor_o/2-1) — meaning one half-period is over.
  - Assert miso_receive_sclk_o flag for one clock cycle of PCLK else, clear it.
  - Else clear it

Generate MOSI Sample Flags

- Create a sequential **block** for mosi_send_sclk_o and mosi_send_sclk0_o.
- On reset, clear both.
- This condition essentially checks whether the **sampling point for MISO data is on the rising or falling edge of SCLK**, based on CPOL and CPHA.

CPOL	CPHA	Active Sampling Edge

0	0	Rising Edge
0	1	Falling Edge
1	0	Falling Edge
1	1	Rising Edge

- If this condition(!cpha_i && cpol_i)||(cpha_i && !cpol_i) is true

  - If the current SCLK is **high** (logic 1).

    - if count_s has reached the final count (BaudRateDivisor_o/2-2) — meaning one half-period is over.
    - Assert mosi_send_sclk_o flag for one clock cycle of PCLK else, clear it.
    - Else clear it

- If the condition (!cpha_i && !cpol_i)||(cpha_i && cpol_i) is true

  - If the current SCLK is **low** (logic 0).

    - if count_s has reached the final count (BaudRateDivisor_o/2-2) — meaning one half-period is over.
    - Assert mosi_send_sclk0_o flag for one clock cycle of PCLK else, clear it.
    - Else clear it

4. SPI Slave Select Generator

Objective: *To write synthesizable RTL code for an SPI salve select generator of the SPI core and verify with a Verilog-based linear testbench . Validate the RTL coding style using the linting process and generate the gate-level netlist.*

Working Directory : /home/<user_name>/spi_design_lab/

Relevant Directories :

tb : tb source code
rtl : rtl source code
sim : Simulation directory
synth : Synthesis directory
lint : Lint directory

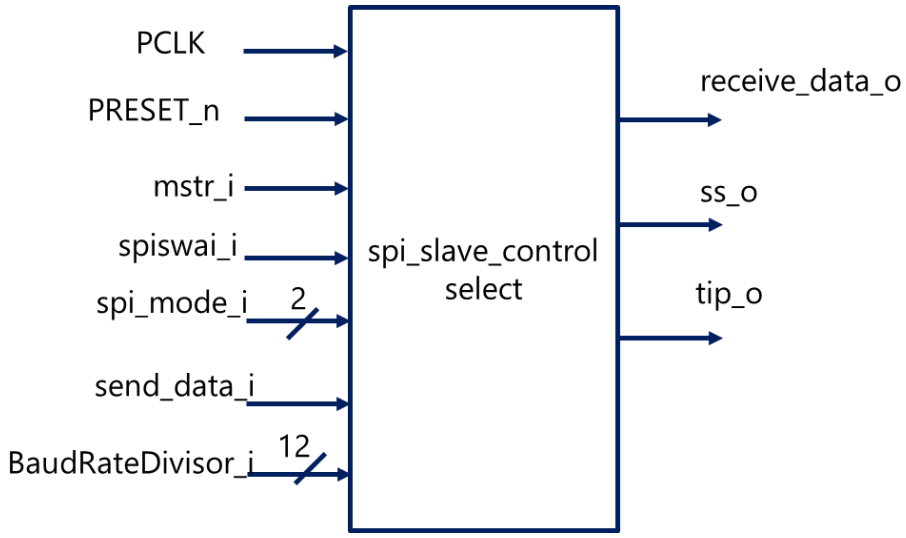
Functionality:

The spi_slave_select module is designed to manage the slave select (SS) signal for a Serial Peripheral Interface (SPI) device in master mode. It ensures the SPI device is appropriately selected and deselected based on data transmission requirements and SPI mode settings. The module operates under the Advanced Peripheral Bus (APB) clock (PCLK) and is responsive to the system reset signal (PRESETn).

The module includes logic to handle different SPI modes, including run and wait modes. When the SPI is in run mode or wait mode (with spiswai not asserted) and a new data transmission is initiated (send_data signal is high), the SS signal is pulled low, indicating the SPI device is selected. A counter is then used to keep SS low for a duration determined by the baud rate divisor (BaudRateDivisor). Once this duration has elapsed, the SS signal is set high again, indicating the end of the SPI transaction.

Additionally, the module generates the receive_data signal to ensure proper data reception. This signal is asserted after the SS signal has been low for the required period, ensuring that the Master In Slave Out (MISO) data is correctly captured. The tip signal is also provided to indicate the transaction in progress status, being high when SS is low. Overall, the spi_slave_select module ensures accurate and reliable SPI device selection and data transfer synchronization in various operating modes.

Block Diagram:



Signal Description:

Signal	Width	Direction	Signal Description
PCLK	1	Input	System Clock
PRESET_n	1	Input	Active Low Asynchronus Reset Signal
mstr_i	1	Input	SPI is in Master Mode
send_data_i	1	Input	Send Data from Data Register
spiswai_i	1	Input	SPI Stop in Wait Mode
spi_mode_i	2	Input	SPI Mode(Run Mode, Wait Mode and Stop Mode)
ss_o	1	Output	Slave Select Active Low Signal
baudratedivisor_i	12	Input	Baud Rate Divisor
tip_o	1	Output	Transfer in Pregress
receive_data_o	1	Output	Receive Data to Data Register

Pseudocode:

Declare Module and Ports

- Name the module: spi_slave_select
- List and define inputs and outputs:
    - Inputs:
        - PRESET_n : Asynchronous active-low reset
        - spi_mode_i[1:0] : Indicates SPI run/wait/stop mode

        - mstr_i : Master mode enable
        - spiswai_i : SPI Wait-In-Stop control
        - PCLK : System clock
        - send_data_i : Indicates new data is written
        - BaudRateDivisor_i[11:0] : Clock divisor for baud rate timing
    - Outputs:
        - ss_o : Slave Select (active-low)
        - receive_data_o : Flag to indicate data reception done
        - tip_o : Transfer-in-progress signal (active high when SS is low)

Declare Internal Signals

- count_s[15:0] : Counter to track baud-based timing
- target_s[15:0] : Stores computed baud-based count value (16 * BaudDivisor)
- rcv_s : Internal register to trigger receive_data_o signal

Continuous Assignments (assign): Derive any **combinational logic expressions** or calculations directly.

- assign target_s to be the product of BaudRateDivisor_i and 16
- assign tip_o as the complement of ss_o to indicate when a transfer is in progress.

Write a sequential logic block sensitive to posedge clock and negedge PRESETn

This block is used to control the slave select signal (ss_o), the counter, and the receive indicator (rcv_s)

- When reset is active (low), initialize count_s to all 1s, assert ss_o by setting it to 1, and clear rcv_s to 0.
- When operating in master mode and either in run mode (spi_mode_i = 00) or in wait mode with spiswai_i low.
    - check if send_data_i is high.
        - If send_data_i is high, assert ss_o to 0 to select the slave and reset the counter to 0.
    - If the counter is less than target_s-1, keep ss_o low, increment the counter.
        - if the counter reaches target_s-1, set rcv_s to 1 to indicate that data reception is complete.
    - If none of these conditions are met, assert ss_o, clear rcv_s, and reset the count_s to all 1’s.
- If the module is not in master mode or an unsupported SPI mode, assert ss_o, clear rcv_s, and reset the count_s to all 1’s

Write a sequential logic block sensitive to posedge clock and negedge PRESETn

This block is used to generate the receive_data_o signal.

- On reset active (low), clear receive_data_o
- Else receive_data_o is registering rcv_s .

5. SPI APB Slave Interface

Objective: *To write synthesizable RTL code for an SPI APB slave interfacer of the SPI core and verify with a Verilog-based linear testbench . Validate the RTL coding style using the linting process and generate the gate-level netlist.*

Working Directory : /home/<user_name>/spi_design_lab/

Relevant Directories :

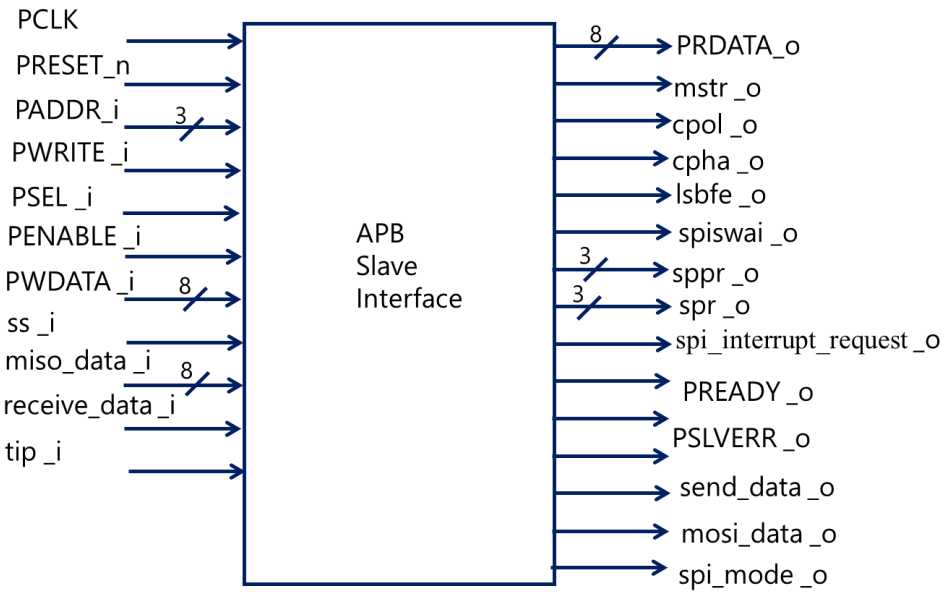
tb : tb source code
rtl : rtl source code
sim : Simulation directory
synth : Synthesis directory
lint : Lint directory

Functionality:

The apb_slave interface is designed to interface with an Advanced Peripheral Bus (APB) and control a Serial Peripheral Interface (SPI) device. It manages APB transactions to read and write SPI registers, including control, status, and data registers. The module uses a state machine with three states (IDLE, SETUP, ENABLE) to handle the APB protocol, ensuring correct timing and data integrity. During the ENABLE state, the module checks if the transaction is a read or write operation and updates the appropriate SPI register accordingly. The module also handles SPI data transmission by sending data to and receiving data from the SPI master.

The module contains logic to generate write and read enable signals based on the APB transaction type and state. It updates SPI control registers with specific masks to ensure only valid bits are modified. The SPI mode state machine, with states for normal operation, wait mode, and stop mode, adjusts the SPI peripheral's operation based on control register settings. Additionally, the module generates an interrupt request signal based on various status flags, providing feedback on the SPI peripheral's status and operation. Overall, the apb_slave interface facilitates seamless communication between the APB bus and the SPI peripheral, enabling efficient data exchange and control.

Blcok diagram:



Signal Description:

Signal	Width	Direction	Signal Description
PCLK	1	Input	System Clock
PRESET_n	1	Input	Active Low Asynchronus Reset Signal
PADDR_i	3	Input	APB Address bus
PSEL_i	1	Input	APB Slave Select Signal
PENABLE_i	1	Input	PENABLE indicates the second and subsequent APB transfer
PWRITE_i	1	Input	Indicates the Write (1) or Read(0).
PWDATA_i	8	Input	The PWDATA is write data bus.
PRDATA_o	8	Output	The PRDATA is read data bus.
PREADY_o	1	Output	PREADY is used to extend an APB Transfer by Core
PSLVERR_o	1	Output	PSLVERR is used to indicate Transfer Error.
ss_i	1	Input	Slave Select (Active Low Signal)
spi_interrupt_request_o	1	Output	SPI Interrupt Request

Signal	Width	Direction	Signal Description
receive_data_i	1	Input	Indicates the transfer of data(MISO data) from Shifter register to data register.
miso_data_i	8	Input	MISO Data from shift register
tip_i	1	Input	Transfer In Progress
send_data_o	1	Output	Indicates the transfer of data from data register to shifter register
mstr_o	1	Output	Indicates Master(1) or Slave (0).
cpol_o	1	Output	Clock Polarity
cpha_o	1	Output	Clock Phase
lsbfe_o	1	Output	LSB First Enable
spiswai_o	1	Output	SPI Stop in Wait Mode
mosi_data_o	8	Output	MOSI Data from Data Register to Shift Register
spi_mode_o	2	Output	SPI Mode(Run Mode, Wait Mode and Stop Mode)
spr_o	3	Output	SPI Baud Rate Selection Bits
sppr_o	3	Output	SPI Baud Rate Preselection Bit

Define Parameter Macros

Use define to declare data width and address width constants for uniformity:

- SPI_APB_DATA_WIDTH of 8 bits
- SPI_REG_WIDTH of 8-bits
- SPI_APB_ADDR_WIDTH of 3 bits

Declare Registers, Wires, and Internal Signals

- Declare:
  - APB State registers (STATE, next_state)-APB FSM
  - SPI Mode state (spi_mode_o, next_mode)- SPI FSM
  - Control/Status/Baud/Data registers (SPI_CR_1, SPI_CR_2, SPI_BR, SPI_SR, SPI_DR)
  - Flags for interrupts and status (spif, sptef, modf,modfen,spe)
  - Write and read enable signals (wr_enb, rd_enb)

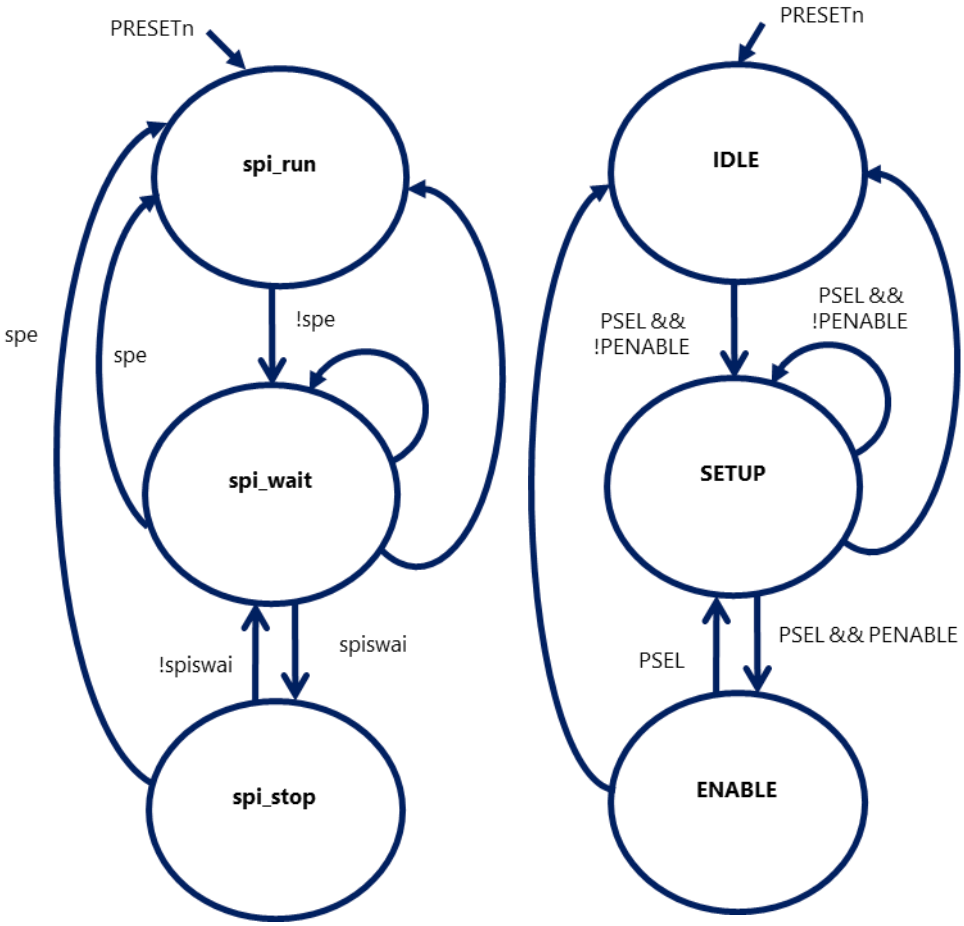
Define Parameters for APB states and SPI modes

Define APB state machine states: IDLE, SETUP, ENABLE

- Define present state(Sequentail logic) and next state logic(Combinational logic)

Define SPI modes: spi_run, spi_wait, spi_stop

- Define present state(Sequential logic) and next state logic(Combinational logic)



Generate APB Control Signals

- Generate:
  - PREADY_i → driven high when in ENABLE state
  - PSLVERR_o → indicates error if tip_i is low in ENABLE state

Generate Write and Read Enables

- wr_enb = High when PWRITE_i is high and in ENABLE state
- rd_enb = High when PWRITE_i is low and in ENABLE state

Implement write operation in SPI registers

- Write a sequential logic to register PWDATA to internal variables(SPI_CR_1, SPI_CR_2, SPI_BR) based on PDDR.
- Mask unused bits of SPI_CR_2, SPI_BR.

Implement APB Read Data Path

- In combinational always @(*)

- If rd_enb is high, select register data based on PADDR_i
- Drive PRDATA_i accordingly
- Else, assign PRDATA_i = 8'b0

Decode Control Register Fields

- Assign fields from SPI_CR_1 to outputs:
  - mstr_o, cpol_o, cpha_o, lsbfe_o, spie, spe, sptie
- Assign fields from SPI_CR_2 to outputs:
  - spiswai_o, modfen
- Assign Baud Rate prescaler values sppr_o, spr_o from SPI_BR

Detect Mode Fault (MODF)

Assign modf high based when slave select ss_i is low and SPI is in master mode and modfen is high and ssoe is low.

Detect spi_interrupt_request_o based on SPI control bits and status flags

spie — SPI interrupt enable

sptie — SPI transmit interrupt enable

spif — SPI transfer complete flag

sptef — SPI transmit buffer empty flag

modf — Mode fault flag

(!spie && !sptie) → no interrupt

(spie && !sptie) → interrupt on spif or modf

(!spie && sptie) → interrupt on sptef

else → interrupt if any of the three flags are 1

Use nested conditional (?:) operators

Update SPI status register (SPI_SR)
Based on:

- Transmit buffer empty flag (SPTEF) is 1 when data register is zero
- SPI transfer complete flag (SPIF) is 1 when data is present in data register.
- Update the status flags SPTEF,SPIF,MODF to SPI_SR.

Write a sequential logic block sensitive to posedge clock and negedge PRESETn

- On reset active (low), clear send_data_o
- Else check for the write enable signal.
  - If wr_enb is high send_data_o = send_data_o
  - Else check the following signals
    - If SPI_DR register is having PWDATA and SPI_DR != miso_data and SPI_mode is either spi_run or spi_wait, send_data_o is made high
    - Else
      - If the spi_mode is either in spi_run or spi_wait and receive data is asserted, then send_data_o is made low
      - Else send_data_o is asserted.

Write a sequential logic block sensitive to posedge clock and negedge PRESETn

- On reset active (low), clear mosi_data_o
- Else check for the write enable signal.
  - If SPI_DR register is having PWDATA and SPI_DR != miso_data and SPI_mode is either spi_run or spi_wait, mosi_data_o is equal to SPI_DR
  - Else
    - Mosi_data_o is equal to its previous value itself (mosi_data_o = mosi_data_o)

Write a sequential logic block sensitive to posedge clock and negedge PRESETn

- On reset active (low), clear the data register SPI_DR
- Else check for the write enable signal.
  - If wr_enb is high
    - Check the Paddr to be equal to 3'b101. If it is equal, Store PWDATA in SPI_DR

        - Else make no changes to the register content , SPI_DR = SPI_DR
    - Else check the following signals
        - If SPI_DR register is having PWDATA and SPI_DR != miso_data and SPI_mode is either spi_run or spi_wait, then clear SPI_DR
        - Else
            - If the spi_mode is either in spi_run or spi_wait and receive data is asserted, then store the mosi_data into SPI_DR
            - Else make no changes to the register content , SPI_DR = SPI_DR

Derive any combinational logic expression for spi_interrupt_request signal

- Spi_int_req is cleared if spie and spite both are low
- Else if spie is high and sptie is low, then spi_int_req is equal to 1 when spif or modf is high
- Else if spite is high and spie is low, then sptef value is stored in spi_int_req
- Else spi_int_req is made high if spif or modf or sptef is asserted.

6. SPI Shifter

Objective: *To write synthesizable RTL code for an SPI shifter of the SPI core and verify with a Verilog-based linear testbench . Validate the RTL coding style using the linting process and generate the gate-level netlist.*

Working Directory : /home/<user_name>/spi_design_lab/

Relevant Directories :
- **tb : tb source code**
- **rtl : rtl source code**
- **sim : Simulation directory**
- **synth : Synthesis directory**
- **lint : Lint directory**

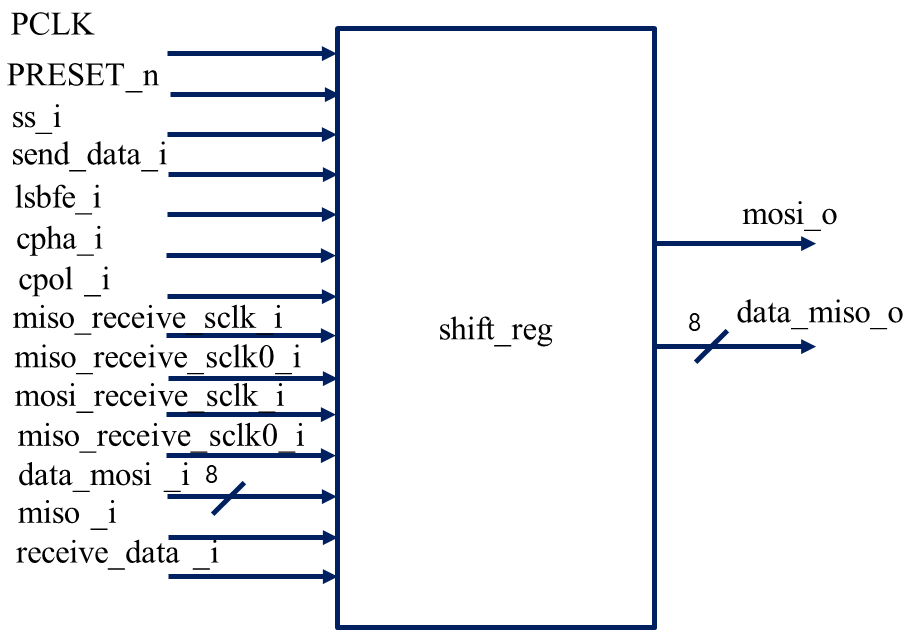
Functionality:

The shifter module is designed to facilitate data transfer in a Serial Peripheral Interface (SPI) by handling the shifting of data in and out of the SPI registers. This module operates under the Advanced Peripheral Bus (APB) clock (PCLK) and is responsible for managing the Master Out Slave In (MOSI) and Master In Slave Out (MISO) data lines during SPI transactions. The module utilizes various configuration parameters such as clock polarity (cpol), clock phase (cpha), and least significant bit first enable (lsbfe) to correctly align the data according to the SPI protocol settings.

The data to be transmitted via MOSI is loaded into a shift register when the send_data signal is asserted. Depending on the SPI configuration, the module uses different always blocks to shift the data out on the MOSI line, synchronized with either the rising or falling edge of the SPI clock (sclk). The shifting process considers both the clock polarity and phase settings to ensure correct timing. Similarly, the module captures data from the MISO line into a temporary register when the receive_data signal is asserted, ensuring that data is properly sampled based on the SPI configuration.

To handle various scenarios, the module employs multiple counters and temporary registers to manage data shifting and sampling for different combinations of clock polarity and phase. The mosi signal is driven based on the combination of cpha and cpol, selecting the appropriate data bit from the shift register. Additionally, the data_miso output is updated with the received data, ensuring proper synchronization. Overall, the shifter module ensures reliable and accurate data transfer in SPI communication by managing the timing and alignment of data bits based on the configured SPI protocol parameters.

Block diagram:



Signal Description:

Signal	Width	Direction	Signal Description
PCLK	1	Input	APB Clock
PRESET_n	1	Input	Active Low Asynchronus Reset Signal
ss_i	1	Input	Slave Select (Active Low Clock)
receive_data_i	1	Input	Indicates the transfer of data(MISO data) from Shifter register to data register.
data_miso_i	8	Output	MISO Data from shift register
send_data_i	1	Input	Indicates the transfer of data from data register to shifter register
miso_i	1	Input	Master In Slave Out.
cpol_i	1	Input	Clock Polarity
cpha_i	1	Input	Clock Phase
lsbfe_i	1	Input	LSB First Enable
mosi_i	1	Output	Master Out Slave In
data_mosi_i	8	Input	MOSI Data from Data Register to Shift Register
miso_receive_sclk_o _o	1	Input	Flag to indicate when to receive the miso data when both cphase and cpol are high or low.
miso_receive_sclk0_o _o	1	Input	Flag to indicate when to receive the miso data when either of the cphase or cpol is high.
mosi_send_sclk_o _o	1	Input	Flag to indicate when to send the mosi data when both cphase and cpol are high or low.
mosi_send_sclk0 _o	1	Input	Flag to indicate when to send the mosi data when either of the cphase or cpol is high.

Declare Internal Signals

- shift_register 8-bitHolds the outgoing 8-bit data to shift on MOSI
- temp_reg 8-bitCollects incoming 8-bit data from MISO
- count, count1 3-Bit counters for MOSI shifting (LSB-first/MSB-first)
- count2, count3 3-Bit counters for MISO shifting (LSB-first/MSB-first)

Transmit Data Register Logic (Shift Register Load)(Sequential logic)

When:

- On reset, clear shift_register
- When send_data=1, load new data_mosi value

Output Received Data

When:

- If receive_data=1, drive data_miso with temp_reg
- Else output 0

Transmit Data Bit-by-Bit (MOSI)

When resetis active then mosi is 1'b0,count is 3'd0,count1is 3'd7.

Else

- SPI slave works only when ss is asserted low
    - Decide Clocking Mode:(!cpha && cpol)||(cpha && !cpol))
        - If LSBFE=1, transmit from bit 0 to bit 7 using count
            - If count <= 7
                - If mosi_send_sclk:Drive MOSI line with correct bit of shift register,Increment the counter
            - Else count is loaded with 0
        - Else

- If count1 >= 0
    - If mosi_send_sclk:Drive MOSI line with correct bit of shift register,Deccrement bit counter.
- Else count is loaded with 0

- Else
    - If LSBFE=1, transmit from bit 0 to bit 7 using count
        - If count <= 7
            - If mosi_send_sclk0:Drive MOSI line with correct bit of shift register,Increment bit counter
        - Else count is loaded with 0
    - Else
        - If count1 >= 0
            - If mosi_send_sclk0:Drive MOSI line with correct bit of shift register,Deccrement bit counter.
        - Else count is loaded with 7

Receive Data Bit-by-Bit (MISO)

When resetis active then mosi is 1'b0,count2 is 3'd0,count3 is 3'd7.

Else

- SPI slave works only when ss is asserted low
    - Decide Clocking Mode:(!cpha && cpol)||(cpha && !cpol))
        - If LSBFE=1, transmit from bit 0 to bit 7 using count
            - If count2 <= 7
                - If miso_recieve_sclk0:Drive MOSI line with correct bit of shift register,Increment the counter
            - Else count is loaded with 0
        - Else
            - If count3 >= 0

- If miso_recieve_sclk0:Drive MOSI line with correct bit of shift register,Deccrement bit counter.

- Else count is loaded with 0

- Else
  - If LSBFE=1, transmit from bit 0 to bit 7 using count
    - If count2 <= 7
      - If miso_recieve_sclk:Drive MOSI line with correct bit of shift register,Increment bit counter
    - Else count is loaded with 0
  - Else
    - If count3 >= 0
      - If miso_recieve_sclk:Drive MOSI line with correct bit of shift register,Deccrement bit counter.
    - Else count is loaded with 7